

Clique duas vezes (ou pressione "Enter") para editar

Strings e bytes

Python faz diferença entre um string e os bytes que o compõem. Assim, diferente de várias linguagens em que um string é um array de bytes, Python os define como elementos de tipos distintos.

Strings

Strings são sequências de caracteres delimitadas por `'`, `"` ou `'''` `"""`. Nesse último caso os strings podem ser digitados em múltiplas linhas.

É possível obter o valor inteiro de cada um dos caracteres associados ao string, usando `ord()`. Para obter o caractere correspondente a um inteiro, usa-se `chr()`

```
meuStr = "Giovanna Camila"
print (meuStr)
for c in meuStr:
    print (c, end='')
print ()

print ('O código do', meuStr[0], 'é', ord(meuStr[0]))

# Concatenando ! ao nome
meuStr += chr(33)
print (meuStr)
```

```
Giovanna Camila
Giovanna Camila
O código do G é 71
Giovanna Camila!
```

bytes

Um array de bytes é expresso similarmente a strings, porém iniciando com **b**. Ou seja `b'`, `b"` ou `b'''` `b"""`. Esse último permite os bytes serem expressos em várias linhas.

Importante: tanto em strings quanto em bytes, caracteres de escape são aceitos. No caso dos strings, são convertidos para o caractere correspondente. Para bytes, o seu valor é mantido.

Algumas sequências de escape:

`\r` - retorno do carro

`\n` - pulo de linha

`\t` - tabulação

`\041` - o caractere (ou valor) corresponde ao octal 0o41 (!)

`\x41` - o caractere (ou valor) corresponde ao hexadecimal 0x41 (A)

Se isso lhe deixa mais confortável, um valor dos tipos bytes ou strings podem ser convertidos em uma lista:

```
lst1 = list(b'ABC')
lst2 = list('ABC')
```

```
meusBytes = b'Meus bytes'
print (meusBytes)
for b in meusBytes:
    print (b, end=' ')
print ()

print (meusBytes[1])

print (list(meusBytes))

meusBytes += b'\41' # concatena '!' e não 'A'
print (meusBytes)
```

```
b'Meus bytes'
77 101 117 115 32 98 121 116 101 115
101
[77, 101, 117, 115, 32, 98, 121, 116, 101, 115]
b'Meus bytes!'
```

Bytes vs ByteArray

A exemplo de *strings*, *bytes* são *read-only*. Isso significa que embora se possa obter os seus elementos usando indexação, **não** é possível fazer atribuições.

Uma alternativa a *bytes* é *bytearray*. É possível criar uma conjunto de bytes com ambos, mas o segundo permite escrita nos elementos.

Veja:

```
ba = bytearray(b'Giovanna')
print (ba)
ba[2] = 65
print (ba)
print (ba[2])
```

Conversão entre Strings e bytes

O método **encode()** converte um string nos bytes correspondentes. Já **decode()** converte bytes em um string. É possível especificar qual o encoding a ser utilizado. A lista de encodings suportados está disponível em:

```
import encodings.aliaes
print (encodings.aliaes.aliaes.values())
```

```
import encodings.aliaes
print (encodings.aliaes.aliaes.values())
```

```
nome = "josé joão da anunciação"
nbytes = nome.encode('utf-8')
```

```
novo_nome = nbytes.decode('cp865')
print (novo_nome)
```

```
dict_values(['ascii', 'ascii', 'ascii', 'ascii', 'ascii', 'ascii', 'ascii', 'ascii', 'ascii', 'ascii', 'ascii', 'ascii', 'base64_codec', 'bas
jos|~ jo|úo da anuncia|º|úo
```

```
s = 'Gio Camila!!!'
emBytes = s.encode()
emBytesUTF = s.encode('utf-8')
print ("Em bytes: ", emBytes)
print ("Em bytes UTF-8: ", emBytesUTF)
```

```
# O string está codificado em bytes, vamos decodificá-lo
novStr = emBytes.decode()
print ("Decodificado: ",novStr)
```

```
novStrUTF = emBytes.decode('utf-8')
print ("Decodificado UTF: ",novStrUTF)
```

```
Em bytes:  b'abc'
Em bytes UTF-8:  b'abc'
Decodificado:  abc
Decodificado UTF:  abc
```

Exibindo valores em Hexa

A formatação de valores hexa pode ser feita com o curinga **x** nos strings de formatação. Lembre que strings de formatação seguem o padrão

```
f"texto {expressao:curinga} texto ..."
```

Alguns curingas de formatação:

Curinga	Formatação
d	decimal
x	hexadecimal
f	float
s	string

O alinhamento de valores numéricos é à direita e, antes de qualquer dos curingas pode-se indicar quantos caracteres devem ser ocupados. Um *zero* antes desses números indicam a exibição de zeros não significativos.

Detalhes aqui: <https://www.python.org/dev/peps/pep-3101/>

```
# Exibição de bytes em hexadecimal
# Ilustra também o uso de comprehensions.
```

```
meusBytes = b'\x09\x41'+b'Gio Camila'
#print (len(meusBytes))
print ("Impressão padrão: ", meusBytes)
print (meusBytes[0], meusBytes[1])
print ("Impressão com for explícito: ", end='0x')
for b in meusBytes:
    print (f"{b:02x}", end='')
print ()

print ("Impressão com comprehensions + join: ", end='')
print("".join([f"{b:02x}" for b in meusBytes]))
```

```
Impressão padrão:  b'\tAGio Camila'
9 65
Impressão com for explícito: 0x094147696f2043616d696c61
Impressão com comprehensions + join: 094147696f2043616d696c61
```

▼ structs em Python

Quando se programa em mais baixo nível, o preciso formato dos dados necessita ser conhecido. O formato do datagrama IP, por exemplo, é:

[Formato do datagrama IP](#)

Observa-se, por exemplo, que o campo do tamanho total do datagrama é expresso por um número de 16 bits (comporta o valor máximo de 65535).

Portanto, **em tese**, se quisermos formar compor um datagrama em Python, no campo *total Length* deveríamos colocar um *int* do Python. **Só em tese**.

Um int do Python ocupa muito mais do que 2 bytes (16 bits). Para saber o tamanho exato, execute o código a seguir.

```
import sys
x = 100
print (f'0 tipo de x é {type(x)} e o tamanho é {sys.getsizeof(x)}')
```

```
0 tipo de x é <class 'int'> e o tamanho é 28
```

Ou seja, um mero inteiro ocupa 28 bytes (esse valor pode variar de sistema para sistema).

A solução é gerar uma estrutura (jargão C), que é similar a uma lista em Python, mas a cada elemento é estabelecido um tamanho pré-definido. Esse processo é o que chamamos de empacotamento (feito via o método `pack()`). Os detalhes podem ser complexos para uma primeira leitura.

A ideia é formar um conjunto de `bytes` em Python, resultado do agrupamos de bits de várias variáveis. É mais fácil com um exemplo:

```
import struct

osBytes = struct.pack ("ii12s", 14, 0xB, "Gio Camila".encode())
print ("Tamanho de osBytes", len(osBytes), " -> ", osBytes)
decodificado = struct.unpack("ii8si", osBytes)
print (decodificado)
decodificado2 = struct.unpack("i", b'\x12\x00\x00\x00')
print (decodificado2)

Tamanho de osBytes 20  ->  b'\x0e\x00\x00\x00\x0b\x00\x00\x00Gio Camila\x00\x00'
(14, 11, b'Gio Cami', 24940)
(18,)
```

Observe que o empacotamento depende de quantos bytes serão reservados para cada elemento. É algo similar à formatação de strings, porém o resultado é do tipo bytes.

Você pode conferir os vários caracteres usados para formatação, aqui: <https://docs.python.org/3/library/struct.html>. Os principais são:

letra	significado
c	char (1 byte)
b	signed char (1 byte)
B	unsigned char (1 byte)
h	short (2 bytes)
H	unsigned short (2 bytes)
i	int (4 bytes)
I	unsigned int (4 bytes)
l	long (4 ou 8 bytes)
L	unsigned long (4 ou 8 bytes)
q	long long (8 bytes)
s	bytes (preceder do número de elementos)

Endianess pode ser controlada precedendo a primeira letra de um dos seguintes símbolos:

Símbolo	Ordem dos bytes
@	nativa
=	nativa
<	little-endian

| big-endian ! | rede (= big-endian)

Da mesma forma que empacotamos um conjunto de variáveis podemos também desempacotar, com `unpack()`. Veja:

```
import struct

osBytes = struct.pack(">ii12s", 14, 0xB, "Gio Camila".encode())
print ("Tamanho de osBytes (empacotamento big-endian):",len(osBytes), " -> ", osBytes)

osBytes = struct.pack ("ii12s", 14, 0xB, "Gio Camila".encode())
print ("Tamanho de osBytes (empacotamento nativo - little-endian)",len(osBytes), " -> ", osBytes)

dia, mes, nome = struct.unpack ("ii12s", osBytes)
print (f"nome: {nome}; aniversário em {dia:2d}/{mes:02d}")
print()

# Observe que o retorno de unpack() é uma tupla, portanto
# é possível usá-lo como generator de um for.

for elem in struct.unpack("ii12s", osBytes):
    print (f"Desempacotado um {type(elem)} com valor: {elem}")
```

Que tal praticarmos?

Os bytes a seguir representam um pacote IP, tal como coletado por um analisador de protocolos. Identifique os vários campos que formam o pacote.

```
pacote = b'\x45\xc0\x00\x50\xea\x14\x00\x00\x01\x59\xd8\x1x6f\x0a\x0a\x0c\xe0\x00\x00\x05\x02\x01\x00\x30\x0a\x0a\x0c\xe0\x00\x00\x00\x00'

print (f"ip-origem: {pacote[12]}.{pacote[13]}.{pacote[14]}.{pacote[15]}")
print (f"ip-destino: {pacote[16]}.{pacote[17]}.{pacote[18]}.{pacote[19]}")
print (f"ttl: {pacote[8]}")

'''
byte0 = f"{pacote[0]:08b}"
print (byte0)
versaoStr = byte0[0:4]
versao = int(versaoStr, 2)
print (versao)
'''

versao = pacote[0] >> 4
print (versao)

#hlen = pacote[0] & ((1 << 4) - 1)
hlen = (pacote[0] & 0xF)
print (hlen)
```

```
ip-origem: 120.54.102.10
ip-destino: 10.12.2.224
ttl: 1
4
5
```

```
dados = bytearray(b'Laura Batista')
key = 112
for i in range (len(dados)):
    dados[i] = dados[i] ^ key
print (dados)

for i in range (len(dados)):
    dados[i] = dados[i] ^ key
print (dados)
```

```
bytearray(b'\x11\x05\x02\x11P2\x11\x04\x19\x03\x04\x11')
bytearray(b'Laura Batista')
```

Que tal praticarmos (2)?

1. Faça um programa que lê o nome de um arquivo qualquer e aplica uma cifra 'xor' em cada *byte* dele, usando um valor que você determina, e gera um novo arquivo. O nome do novo arquivo é o nome original acrescido de '.enc'.
2. Faça um programa que decifra o arquivo acima.

